Capstone Project INF 331
NFL Team Performance Analyisis

By Jadyn McClaude

Research Question

Dataset: NFL team stats (2003-2023)

Question: Do better offensive stats (points, yards, turnovers) lead to more wins

Data Cleaning

- Removed Missing Values
- Standardized column names(lowercase, and underscores
- Converted data types (strings into intengers
- Cleaned formatting issues

Cleaning made the dataset consistent and ready for analysis

Data Cleaning Explained part 1

Fixed Column names

- I standardized column names by removing spaces and making easier to work with in SQL and Python
- Removed extra spaces
- Made everything lowercase
- Replaces spaces with underscores

Checked Missing values df.isnull().sum()

- I checked for missing values to identify any incomplete data
- Pretty much all columns had values
- The column ties was sometimes null

```
memory usage: 183.9 KB

[7]: df.columns = df.columns.str.strip().str.lower().str.replace("

     print("Cleaned Column Names:")
     print(df.columns)

Cleaned Column Names:
Index(['year', 'team', 'wins', 'losses', 'win_loss_perc', 'poi
nts',
       'points_opp', 'points_diff', 'mov', 'g', 'total_yards',
'plays_offense',
       'yds_per_play_offense', 'turnovers', 'fumbles_lost', 'f
irst_down',
       'pass_cmp', 'pass_att', 'pass_yds', 'pass_td', 'pass_in
t',
       'pass_net_yds_per_att', 'pass_fd', 'rush_att', 'rush_yd
s', 'rush_td',
       'rush_yds_per_att', 'rush_fd', 'penalties', 'penalties_
yds', 'pen_fd',
       'score_pct', 'turnover_pct', 'exp_pts_tot', 'ties'],
      dtype='str')

[8]: print("Missing values per column:")
     print(df.isnull().sum())

Missing values per column:
year                      0
team                      0
wins                      0
losses                    0
win_loss_perc             0
points                    0
points_opp                0
points_diff               0
mov                     320
g                         0
total_yards               0
plays_offense             0
yds_per_play_offense      0
turnovers                 0
fumbles_lost              0
first_down                0
pass_cmp                  0
pass_att                  0
pass_yds                  0
pass_td                   0
pass_int                  0
pass_net_yds_per_att      0
pass_fd                   0
rush_att                  0
```

Data Cleaning Part 2

Removed duplicates (df.drop_duplicates)

- I removed duplicate records to ensure each team-season entry is unique.
- Makes sure no rows repeat
- Keeps Dataset Accurate

Saved Cleaned Dataset (df.to-csv)

- Creates final clean dataset for PostgreSQL

```
[9]: df = df.drop_duplicates()
     print("Dataset size after removing duplicates:", df.shape)

     Dataset size after removing duplicates: (672, 35)

[10]: df.to_csv(
      "/Users/jadyn/Desktop/Databases class/nfl-team-analysis/data/nfl_team_stats_cleaned.csv",
      index=False
      )

      print("Cleaned CSV file saved successfully!")

      Cleaned CSV file saved successfully!
```

After cleaning the dataset, a new CSV file is created to store the processed data. This cleaned dataset will be used for loading the data into the PostgreSQL database and for further analysis in the project.

PostgreSQL DataBase Design

- I split the original dataset into 3 normalized tables: teams, seasons, and offensive_stats.
- Primary keys: team_id, season_id, and offense_id.
- Foreign keys: seasons.team_id connects to teams.team_id, and offensive_stats.season_id connects to seasons.season_id.
- One team can have many seasons, so teams → seasons is a one-to-many relationship.
- This design avoids one giant table and makes JOIN queries easier for analysis.

```
cursor.execute("DROP TABLE IF EXISTS teams;"

cursor.execute("""
CREATE TABLE teams (
    team_id SERIAL PRIMARY KEY,
    team_name TEXT UNIQUE
);
""")

conn.commit()
print("Teams table created successfully!")
```

```
Teams table created successfully!
```

```
cursor.execute("DROP TABLE IF EXISTS seasons;")

cursor.execute("""
CREATE TABLE seasons (
    season_id SERIAL PRIMARY KEY,
    year INT,
    team_id INT REFERENCES teams(team_id),
    wins INT,
    losses INT,
    win_loss_perc FLOAT,
    ties INT
);
""")

conn.commit()
print("Seasons table created successfully!")
```

```
Seasons table created successfully!
```

```
cursor.execute("DROP TABLE IF EXISTS offensive_stats;")

cursor.execute("""
CREATE TABLE offensive_stats (
    offense_id SERIAL PRIMARY KEY,
    season_id INT REFERENCES seasons(season_id),
    points INT,
    total_yards INT,
    plays_offense INT,
    yds_per_play_offense FLOAT,
    turnovers INT,
    pass_yds INT,
    pass_td INT,
    rush_yds INT,
    rush_td INT
);
""")

conn.commit()
print("Offensive stats table created successfully!")
```

```
Offensive stats table created successfully!
```

SQL INSIGHTS (Join Query Points Vs Wins)

What it does: Joins teams, seasons, and offensive_stats to compare points scored with wins.

Key result: Teams with higher points usually had more wins.

Connection: This supports the idea that stronger offense is linked to better records.

```
5]: # Query 1 (offense vs wins)
    #Does a high scoring offense mean a good season?

    cursor.execute("""
    SELECT t.team_name, s.year, s.wins, s.losses, o.points, o.total_yards
    FROM offensive_stats o
    JOIN seasons s ON o.season_id = s.season_id
    JOIN teams t ON s.team_id = t.team_id
    ORDER BY o.points DESC
    LIMIT 10;
    """)

    print(cursor.fetchall())
```

```
[('Denver Broncos', 2013, 13, 3, 606, 7317), ('New England Patriots', 2007, 16, 0, 589, 6580), ('Kansas City Chiefs', 2018, 12, 4, 565, 6810), ('Green Bay Packers', 2011, 15, 1, 560, 6482), ('New England Patriots', 2012, 12, 4, 557, 6846), ('New Orleans Saints', 2011, 13, 3, 547, 7474), ('Atlanta Falcons', 2016, 11, 5, 540, 6653), ('Baltimore Ravens', 2019, 14, 2, 531, 6521), ('Dallas Cowboys', 2021, 12, 5, 530, 6919), ('Los Angeles Rams', 2018, 13, 3, 527, 6738)]
```

Teams with higher points scored tend to have more wins.
This suggests that strong offensive scoring is linked to successful seasons.

SQL Insights (Group By Query Average Points By team)

What it does: Groups by team and calculates average points scored.

Key result: Some teams consistently had higher scoring offenses over time.

Connection: This helps show which teams were offensively strong across multiple seasons.

```
#Query 4 (Average points by Team)

cursor.execute("""
SELECT t.team_name, ROUND(AVG(o.points), 2) AS avg_points
FROM offensive_stats o
JOIN seasons s ON o.season_id = s.season_id
JOIN teams t ON s.team_id = t.team_id
GROUP BY t.team_name
ORDER BY avg_points DESC
LIMIT 10;
""")

print(cursor.fetchall())
```

```
[('New England Patriots', Decimal('432.52')), ('New Orleans Saints', Decimal('417.14')), ('Green Bay Packers', Decimal('411.81')), ('San Diego Chargers', Decimal('403.21')), ('Indianapolis Colts', Decimal('397.62')), ('Philadelphia Eagles', Decimal('396.24')), ('Los Angeles Rams', Decimal('395.75')), ('Dallas Cowboys', Decimal('394.24')), ('Los Angeles Chargers', Decimal('387.86')), ('Kansas City Chiefs', Decimal('387.10'))]
```

This query finds the average points scored by each team.

Teams with higher average points tend to have stronger offenses overall.

SQL insights Case Query Team Performance Level

What it does: Labels teams as “Great,” “Average,” or “Poor” based on wins.

Key result: Teams with higher win totals fall into stronger performance categories.

Connection: This helps compare offensive stats against overall team success.

[49]:
```
#Case Query Idea (Team Perfomance Level)

cursor.execute("""
SELECT t.team_name, s.year, s.wins, s.losses,
CASE
    WHEN s.wins >= 12 THEN 'Great Season'
    WHEN s.wins >= 8 THEN 'Average Season'
    ELSE 'Poor Season'
END AS performance_level
FROM seasons s
JOIN teams t ON s.team_id = t.team_id
ORDER BY s.wins DESC
LIMIT 15;
""")

print(cursor.fetchall())
```

```
[('New England Patriots', 2007, 16, 0, 'Great Season'), ('Green Bay Packers', 2011, 15, 1, 'Great Season'), ('Carolina Panthers', 2015,
15, 1, 'Great Season'), ('Pittsburgh Steelers', 2004, 15, 1, 'Great Season'), ('New England Patriots', 2016, 14, 2, 'Great Season'), ('K
ansas City Chiefs', 2020, 14, 2, 'Great Season'), ('Kansas City Chiefs', 2022, 14, 3, 'Great Season'), ('New England Patriots', 2004, 1
4, 2, 'Great Season'), ('New England Patriots', 2010, 14, 2, 'Great Season'), ('San Diego Chargers', 2006, 14, 2, 'Great Season'), ('New
England Patriots', 2003, 14, 2, 'Great Season'), ('Indianapolis Colts', 2009, 14, 2, 'Great Season'), ('Indianapolis Colts', 2005, 14,
2, 'Great Season'), ('Baltimore Ravens', 2019, 14, 2, 'Great Season'), ('Philadelphia Eagles', 2022, 14, 3, 'Great Season')]
```

This query groups teams based on their number of wins.

It shows that teams with more wins are classified as great seasons, while teams with fewer wins are considered poor seasons.

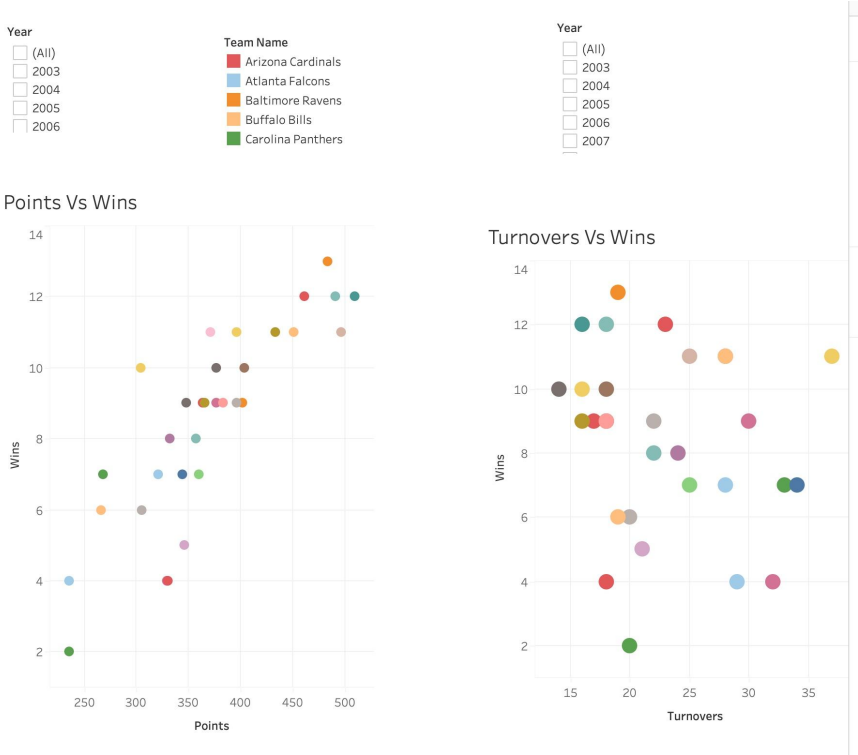
Live Demonstration of My API

I used FastAPI and connected it to my PostgreSQL database, It is pulling real records from my teams, seasons, and offensive_stats tables.

“My first endpoint is GET /items, which returns multiple NFL team-season records from the database.

“My second endpoint is GET /items/{team_name}.

“My POST endpoint allows data to be sent to the API.

Tableau Dashboard

Teams that score more points tend to win more games, while teams with more turnovers generally win fewer games.